

NLP Lab 4 Reference Manual: PyTorch Sequence Models & Code Walkthrough

Part 1: PyTorch Essentials for Sequence Processing

1. Core Tensors & Device Management

PyTorch operates on **Tensors**, which are multidimensional arrays optimized for GPU acceleration and automatic differentiation.

```
# Check hardware availability and create a dynamic device target
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# Moving tensors to hardware device
x = torch.tensor([[1, 2, 3], [4, 5, 6]], dtype=torch.long).to(device)
```

Key Rule for Beginners: Every tensor passed into a PyTorch model (including model parameters themselves) **must reside on the same device**. Mismatched devices (**cpu** vs **cuda:0**) will trigger runtime errors.

2. Structuring Neural Networks (**nn.Module**)

Every custom model in PyTorch inherits from **nn.Module**. You must define:

1. **__init__()**: Instantiates trainable layers (e.g., embeddings, LSTMs, linear layers).
2. **forward()**: Defines the computational graph execution path.

```
class SimpleClassifier(nn.Module):

    def __init__(self, vocab_size, hidden_dim):

        super().__init__()

        self.embedding = nn.Embedding(vocab_size, 300)

        self.fc = nn.Linear(300, hidden_dim)

    def forward(self, x):

        # x shape: (batch_size, seq_len)

        embedded = self.embedding(x)  # shape: (batch_size, seq_len,
300)
```

```

        out = self.fc(embedded)          # shape: (batch_size, seq_len,
hidden_dim)

    return out

```

3. The PyTorch Optimization Pipeline

Training loop execution in PyTorch requires explicit management of gradients:

```

optimizer = optim.Adam(model.parameters(), lr=0.001)

criterion = nn.CrossEntropyLoss()

# Training iteration pattern

optimizer.zero_grad()    # 1. Clear residual gradients from prior
iteration

outputs = model(inputs)  # 2. Perform forward pass

loss = criterion(outputs, targets) # 3. Compute loss value

loss.backward()          # 4. Compute partial derivatives via
backpropagation

optimizer.step()         # 5. Update weights using computed gradients

```

Built-in PyTorch Functionalities Used in Lab 4

1. Pretrained Embeddings (**nn.Embedding.from_pretrained**)

Loads external vectors (like Google News Word2Vec or FastText) into a PyTorch embedding layer.

```

self.embedding = nn.Embedding.from_pretrained(

    embed_matrix, # Pretrained matrix tensor (vocab_size, embed_dim)

    freeze=False, # Set to False to allow fine-tuning during
training

    padding_idx=0 # Forces gradient updates for index 0 (<PAD>) to
remain zero

)

```

2. Recurrent Layers (**nn.RNN** and **nn.LSTM**)

PyTorch recurrent units process 3D sequences. Always set `batch_first=True` for intuitive tensor indexing: (`batch_size`, `sequence_length`, `feature_dimension`).

- **nn.RNN Output:**
 - **output:** Sequence of hidden states across all time steps $\rightarrow$ (`batch`, `seq_len`, `hidden_dim * num_directions`).
 - **h_n:** Final hidden state for each element in the batch $\rightarrow$ (`num_layers * num_directions`, `batch`, `hidden_dim`).
- **nn.LSTM Output:**
 - **output:** Sequence of outputs for all steps.
 - (**h_n**, **c_n**): Tuple containing final **hidden states** (h_n) and **cell states** (c_n).
 -

```
# Bidirectional 2-layer LSTM definition

self.lstm = nn.LSTM(

    input_size=300,

    hidden_size=128,

    num_layers=2,

    bidirectional=True,

    batch_first=True

)
```

3. Loss Masking (**ignore_index**)

When working with variable-length sequences, padded locations (<PAD> tokens mapped to index 0) must not influence loss calculation or gradient updates.

```
# Ignores loss calculation for target targets with value = 0

criterion = nn.CrossEntropyLoss(ignore_index=0)
```

4. Batch Matrix Multiplication (**torch.bmm**)

Computes batch-wise matrix products without flattening or looping. Used extensively in **Attention Mechanisms**:

Given:

- Query $Q \in \mathbb{B} \times 1 \times h$
- Key $K \in \mathbb{B} \times S \times h$

```
# Transpose Keys to (batch, hidden_dim, seq_len)
```

```
keys_transposed = K.transpose(1, 2)
```

```
# Batch Multiply: (batch, 1, hidden_dim) x (batch, hidden_dim, seq_len) -> (batch, 1, seq_len)
```

```
attention_scores = torch.bmm(Q, keys_transposed)
```

5. Sampling Methods (**torch.topk** & **torch.multinomial**)

Used in **Autoregressive Text Generation** to restrict predictions to reasonable candidate tokens:

```
# Extract top-K logit values and their corresponding indices
```

```
top_k_logits, top_k_indices = torch.topk(logits, k=5, dim=-1)
```

```
# Sample a single token index from a probability distribution tensor
```

```
sampled_index = torch.multinomial(probs_distribution, num_samples=1)
```

Part 2: Code Walkthrough

Topic 0: Pretrained Embedding Integration (Word2Vec to PyTorch)

1. Initializing Special Tokens & Storage

```
# Initialize vocabulary with PAD and UNK tokens
```

```
vocab = {"<PAD>": 0, "<UNK>": 1}
```

```
# Index 0 -> Zeros for PAD; Index 1 -> Random normal vector for UNK

embedding_weights = [np.zeros(embedding_dim),
np.random.randn(embedding_dim)]
```

- **Explanation:** Index 0 is assigned to <PAD> (used for padding shorter sentences) and mapped to a zero vector so it contributes zero magnitude initially. Index 1 is reserved for <UNK> (out-of-vocabulary words) and initialized with a random Gaussian vector.

2. Mapping Corpus Tokens to Gensim Vectors

```
for sentence in corpus:

    for word in sentence:

        if word not in vocab:

            vocab[word] = len(vocab)

            if word in w2v_model:

                embedding_weights.append(w2v_model[word])

            else:

embedding_weights.append(np.random.randn(embedding_dim))
```

- **Explanation:** Iterates over every word in the dataset. If the word exists in the pre-trained Google Word2Vec model, its pre-trained 300-dimensional vector is extracted. If the word is missing from Google Word2Vec, a random normal vector is generated.

3. Converting to a PyTorch Tensor

```
embedding_matrix = torch.tensor(np.array(embedding_weights),
dtype=torch.float32)
```

- **Explanation:** Converts the list of NumPy array vectors into a 2D PyTorch FloatTensor of shape (vocab_size, 300) ready to be loaded into neural network layers.

Topic 1: Sequence Classification (Vanilla RNN vs. Stacked BiLSTM)

1. Sequence Encoding & Truncation/Padding

```
def encode(sent, vocab, max_len=7):  
    tokens = [vocab.get(w, vocab["<UNK>"]) for w in sent.split()]  
    return tokens[:max_len] + [vocab["<PAD>"]] * max(0, max_len - len(tokens))
```

- **Explanation:** Transforms raw text sentences into integer sequences. Truncates long sequences to **max_len=7** or appends **<PAD>** (index 0) to short sequences to enforce uniform batch shapes.

2. Vanilla RNN Classifier Architecture

```
class VanillaRNNClassifier(nn.Module):  
    def __init__(self, embed_matrix, hidden_dim):  
        super().__init__()   
        self.embedding = nn.Embedding.from_pretrained(embed_matrix, freeze=False, padding_idx=0)  
        self.rnn = nn.RNN(embed_matrix.shape[1], hidden_dim, batch_first=True)  
        self.fc = nn.Linear(hidden_dim, 1)  
  
    def forward(self, x):  
        embedded = self.embedding(x)  
        out, h_n = self.rnn(embedded)  
        return self.fc(h_n.squeeze(0)).squeeze(1)
```

- **nn.Embedding.from_pretrained(..., freeze=False, padding_idx=0):** Loads the Word2Vec matrix. **freeze=False** enables fine-tuning during training, while **padding_idx=0** prevents index 0 (**<PAD>**) from receiving gradient updates.

- **self.rnn(...)**: Processes embedded sequences step-by-step. **batch_first=True** expects shapes of (batch_size, sequence_length, embedding_dim).
- **h_n.squeeze(0)**: Extracts the final hidden state vector across time steps and passes it to a dense linear layer for binary sentiment output.

3. Stacked Bidirectional LSTM Architecture

```
class StackedBiLSTMClassifier(nn.Module):

    def __init__(self, embed_matrix, hidden_dim, num_layers=2):

        super().__init__()

        self.embedding = nn.Embedding.from_pretrained(embed_matrix,
freeze=False, padding_idx=0)

        self.lstm = nn.LSTM(

            embed_matrix.shape[1], hidden_dim,

            num_layers=num_layers, batch_first=True,
bidirectional=True

        )

        self.fc = nn.Linear(hidden_dim * 2, 1)

    def forward(self, x):

        embedded = self.embedding(x)

        out, (h_n, c_n) = self.lstm(embedded)

        forward_h = h_n[-2, :, :]

        backward_h = h_n[-1, :, :]

        bi_hidden = torch.cat((forward_h, backward_h), dim=1)

        return self.fc(bi_hidden).squeeze(1)
```

- **num_layers=2, bidirectional=True**: Stacked 2-layer LSTM where processing occurs both forward (left-to-right) and backward (right-to-left).
- **h_n[-2, :, :] and h_n[-1, :, :]**: Retrieves the final hidden states from the top layer for both forward (-2) and backward (-1) passes.

- **torch.cat(..., dim=1)**: Concatenates both directions into a single representation vector of size `hidden_dim * 2` before classification.

4. Binary Classification Loss & Optimizer Setup

```
criterion = nn.BCEWithLogitsLoss()
```

```
opt_rnn = optim.Adam(rnn_model.parameters(), lr=0.01)
```

- **nn.BCEWithLogitsLoss()**: Combines a Sigmoid layer and Binary Cross-Entropy loss in a single layer for numerical stability.

Topic 2: Sequence Labeling with Bidirectional RNN

1. Bidirectional POS Tagging Network

```
class BiRNNSequenceLabeler(nn.Module):

    def __init__(self, embed_matrix, hidden_dim, tagset_size):

        super().__init__()

        self.embedding = nn.Embedding.from_pretrained(embed_matrix,
freeze=False, padding_idx=0)

        self.rnn = nn.RNN(embed_matrix.shape[1], hidden_dim,
batch_first=True, bidirectional=True)

        self.fc = nn.Linear(hidden_dim * 2, tagset_size)

    def forward(self, x):

        embeds = self.embedding(x)

        rnn_out, _ = self.rnn(embeds) # (batch, seq_len, hidden_dim *
2)

        logits = self.fc(rnn_out)      # (batch, seq_len, tagset_size)

        return logits
```

- **Explanation:** Unlike classification (which returns one vector per sequence), sequence labeling outputs predictions for *every token step*. `rnn_out` maintains sequence dimension `seq_len`, and `self.fc` projects each token representation to the POS tag set size.

2. Loss Masking with `ignore_index`

```
pos_criterion = nn.CrossEntropyLoss(ignore_index=0)
```

```
# Inside Training Loop:
```

```
logits = pos_model(X_pos)
```

```
loss = pos_criterion(logits.view(-1, len(tag_to_ix)), y_pos.view(-1))
```

- **`logits.view(-1, tagset_size)`:** Flattens 3D output tensors (`batch`, `seq_len`, `tagset_size`) into 2D tensors to match standard Cross-Entropy requirements.
- **`ignore_index=0`:** Automatically ignores loss computation and gradient backpropagation for `<PAD>` tokens.

Topic 3: Language Modeling & Sentence Generation with Stacked LSTM

1. Data Structuring via Sliding Window

```
seq_length = 5
```

```
lm_X, lm_Y = [], []
```

```
encoded_words = [vocab.get(w, vocab["<UNK>"]) for w in words]
```

```
for i in range(0, len(encoded_words) - seq_length):
```

```
    lm_X.append(encoded_words[i : i + seq_length])
```

```
    lm_Y.append(encoded_words[i + 1 : i + seq_length + 1])
```

- **Explanation:** Constructs an autoregressive sequence dataset. For every sequence slice of length 5 (`lm_X`), the target label sequence (`lm_Y`) is shifted forward by exactly 1 token position.

2. Stacked LSTM Language Model

```
class StackedLSTMLanguageModel(nn.Module):

    def __init__(self, embed_matrix, hidden_dim=128, num_layers=2):

        super().__init__()

        self.embedding = nn.Embedding.from_pretrained(embed_matrix,
freeze=False, padding_idx=0)

        embed_dim = embed_matrix.shape[1]

        self.lstm = nn.LSTM(embed_dim, hidden_dim,
num_layers=num_layers, batch_first=True)

        self.fc = nn.Linear(hidden_dim, embed_matrix.shape[0])

    def forward(self, x, hidden=None):

        embeds = self.embedding(x)

        out, hidden = self.lstm(embeds, hidden)

        logits = self.fc(out)

        return logits, hidden
```

- **Explanation:** Takes input sequences and outputs predictions over the entire vocabulary (`embed_matrix.shape[0]`) for each token position. Passes hidden state tuples across time steps during evaluation/generation.

3. Autoregressive Generation with Temperature Sampling

```
with torch.no_grad():

    for _ in range(generate_length):

        logits, hidden = model(input_eval, hidden)

        # Scale logits using temperature
```

```

last_logits = logits[0, -1, :] / temperature

probs = torch.softmax(last_logits, dim=-1)

# Sample index based on probability distribution

pred_idx = torch.multinomial(probs, num_samples=1).item()

# Prepare input for next step

input_eval = torch.tensor([[pred_idx]],
dtype=torch.long).to(device)

generated_words.append(idx2word.get(pred_idx, "<UNK>"))

```

- **logits[0, -1, :] / temperature:** Extracts predictions for the last predicted word position and divides by **temperature** (lower values make distribution sharper/more deterministic; higher values increase diversity).
- **torch.multinomial(probs, num_samples=1):** Samples the next token randomly according to the Softmax probability distribution.

Topic 4: Encoder-Decoder Seq2Seq Machine Translation

1. Building English Word2Vec Matrix for Encoder

```

en_embedding_matrix = torch.zeros((len(en_vocab), 300))

for word, idx in en_vocab.items():

    if word in w2v_google:

        en_embedding_matrix[idx] = torch.tensor(w2v_google[word])

    else:

        if word != "<PAD>":

            en_embedding_matrix[idx] = torch.randn(300) * 0.1

```

- **Explanation:** Creates a specialized embedding tensor for the source language (English) using pre-trained Word2Vec vectors. Unseen English tokens are randomly initialized.

2. Encoder Architecture

```
class Seq2SeqEncoder(nn.Module):  
  
    def __init__(self, embed_matrix, hidden_dim):  
  
        super().__init__()  
  
        self.embedding = nn.Embedding.from_pretrained(embed_matrix,  
freeze=False, padding_idx=0)  
  
        self.lstm = nn.LSTM(embed_matrix.shape[1], hidden_dim,  
batch_first=True)  
  
  
    def forward(self, x):  
  
        embedded = self.embedding(x)  
  
        _, (hidden, cell) = self.lstm(embedded)  
  
        return hidden, cell
```

- **Explanation:** Processes the source English text. Discards sequence outputs and returns only final hidden (**hidden**) and cell (**cell**) states, which act as context vectors containing sentence information.

3. Decoder Architecture

```
class Seq2SeqDecoder(nn.Module):  
  
    def __init__(self, vocab_size, embed_dim, hidden_dim):  
  
        super().__init__()  
  
        self.embedding = nn.Embedding(vocab_size, embed_dim,  
padding_idx=0)  
  
        self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True)  
  
        self.fc = nn.Linear(hidden_dim, vocab_size)  
  
  
    def forward(self, x, hidden, cell):
```

```

embedded = self.embedding(x)

out, (hidden, cell) = self.lstm(embedded, (hidden, cell))

preds = self.fc(out)

return preds, hidden, cell

```

- **Explanation:** Processes destination Bengali tokens step-by-step. Receives initial context state vectors (**hidden, cell**) from the Encoder and updates them sequentially as translated words are generated.

4. Seq2Seq Parent Model with Teacher Forcing

```

class Seq2SeqTranslation(nn.Module):

    def forward(self, src, trg):

        batch_size = src.shape[0]

        trg_len = trg.shape[1]

        trg_vocab_size = self.decoder.fc.out_features

        outputs = torch.zeros(batch_size, trg_len,
                               trg_vocab_size).to(device)

        # 1. Pass English sentence through encoder
        hidden, cell = self.encoder(src)

        # 2. First decoder input is <SOS> token
        decoder_input = trg[:, 0].unsqueeze(1)

        # 3. Teacher forcing loop
        for t in range(1, trg_len):

```

```

        output, hidden, cell = self.decoder(decoder_input, hidden,
cell)

        outputs[:, t, :] = output.squeeze(1)

        decoder_input = trg[:, t].unsqueeze(1) # Feed actual
ground-truth token

    return outputs

```

- **Teacher Forcing Mechanism:** During training, instead of feeding the model's own (potentially incorrect) previous prediction, `decoder_input = trg[:, t].unsqueeze(1)` feeds the actual target Bengali word from the ground-truth data at step `t` to stabilize training convergence.

5. Translation Inference Loop

```

with torch.no_grad():

    hidden, cell = model.encoder(src_tensor)

    decoder_input = torch.tensor([[bn_vocab["<SOS>"]]],
dtype=torch.long).to(device)

    translated_words = []

    for _ in range(max_bn_len):

        output, hidden, cell = model.decoder(decoder_input, hidden,
cell)

        pred_idx = output.argmax(dim=-1).item()

        if pred_idx == bn_vocab["<EOS>"]:

            break

    translated_words.append(inv_bn_vocab.get(pred_idx, ""))

```

```
decoder_input = torch.tensor([[pred_idx]],
dtype=torch.long).to(device)
```

- **Explanation:** During translation testing/inference, teacher forcing is disabled. The model feeds its own argmax prediction (`pred_idx`) as the input token to the next step until `<EOS>` (end of sentence) is generated.

Task / Topic	Model Class	Inputs / Shapes	Key Operations & PyTorch Functions
Topic 0: Pretrained Vectors	<code>build_pretrained_embedding_matrix()</code>	Corpus tokens → Tensor	<code>api.load()</code> , <code>torch.tensor(..., dtype=torch.float32)</code>
Topic 1: Classification	<code>VanillaRNNClassifier</code>	(B, T) token IDs	<code>nn.RNN</code> , <code>h_n.squeeze(0)</code> , <code>nn.BCEWithLogitsLoss()</code>
Topic 1: Classification	<code>StackedBiLSTMClassifier</code>	(B, T) token IDs	<code>nn.LSTM(num_layers=2, bidirectional=True)</code> , <code>torch.cat((h_fwd, h_bwd), dim=1)</code>
Topic 2: Tagging	<code>BiRNNSequenceLabeler</code>	(B, T) token IDs	<code>nn.RNN(bidirectional=True)</code> , <code>logits.view(-1, C)</code> , <code>CrossEntropyLoss(ignore_index=0)</code>
Topic 3: LM Generation	<code>StackedLSTMLanguageModel</code>	(B, T) sliding window	<code>nn.LSTM(num_layers=2)</code> , <code>torch.multinomial(probs, 1)</code>
Topic 4: Translation	<code>Seq2SeqTranslation</code>	Parallel src/trg Tensors	Teacher Forcing iteration, <code>Encoder(h, c)</code> context passing